

OPEN TECH NOTES

loạt tài liệu kỹ thuật miễn phí cho cộng đồng

FOUNDATION MODELS

Nền tảng của làn sóng AI hiện đại

Cách mô hình nền tảng được huấn luyện, căn chỉnh, và lựa chọn cho ứng dụng thực tế

Phần 1 trong loạt bài Kỹ thuật AI Engineering

Biên soạn cho cộng đồng IT Việt Nam · Tháng 8, 2026

Mục lục

Mở đầu	3
1. Foundation model là gì	3
2. Vòng đời: từ pretraining đến trợ lý biết nghe lời	4
2.1. Pretraining giai đoạn tốn kém nhất	4
2.2. Post-training / Alignment dạy mô hình "biết nghe lời"	5
3. Hai hướng kiến trúc: Dense và Mixture-of-Experts	5
3.1. Dense: đơn giản, dễ dự đoán	5
3.2. Mixture-of-Experts: tách dung lượng khỏi chi phí suy luận	6
4. Chọn mô hình cho bài toán thực tế	6
4.1. Closed API và open-weight	7
4.2. Cửa sổ ngữ cảnh và đa phương thức	7
4.3. Chi phí, độ trễ, và "đủ tốt" thay vì "tốt nhất"	7
5. Checklist nhanh cho đội ngũ	7
Đọc thêm	8
Tài liệu tham khảo	8

Mở đầu

Ba năm trở lại đây, gần như mọi sản phẩm phần mềm mới đều nhắc đến "AI" ở đâu đó trong câu chuyện của mình. Nhưng phần lớn các sản phẩm đó không tự huấn luyện mô hình từ đầu chúng được xây trên nền của một nhóm nhỏ các mô hình khổng lồ, được gọi chung là foundation model (mô hình nền tảng): những mạng nơ-ron được huấn luyện trên khối lượng dữ liệu văn bản, mã nguồn, hình ảnh khổng lồ, rồi được các đội kỹ sư khác tái sử dụng cho hàng ngàn bài toán khác nhau, từ chatbot chăm sóc khách hàng đến trợ lý viết code, từ tóm tắt tài liệu pháp lý đến hệ thống RAG tra cứu nội bộ.

Tài liệu này là phần mở đầu của loạt bài Kỹ thuật AI Engineering biên soạn cho cộng đồng kỹ sư phần mềm Việt Nam, giúp bạn có một bức tranh vững chắc về foundation model: nó được tạo ra như thế nào, các dạng kiến trúc phổ biến hiện nay, và những tiêu chí thực tế để chọn đúng mô hình cho bài toán của mình thay vì chọn theo tên tuổi hay theo trào lưu.

Nội dung được tổng hợp và diễn giải bằng hiểu biết chung của ngành, có trích dẫn nguồn công khai ở cuối tài liệu. Đây không phải là bản dịch hay tóm tắt của bất kỳ cuốn sách trả phí nào nếu bạn muốn đào sâu hơn nữa với một góc nhìn hệ thống, đầy đủ case study thực chiến, phần "Đọc thêm" ở cuối tài liệu có gợi ý cụ thể.

1. Foundation model là gì

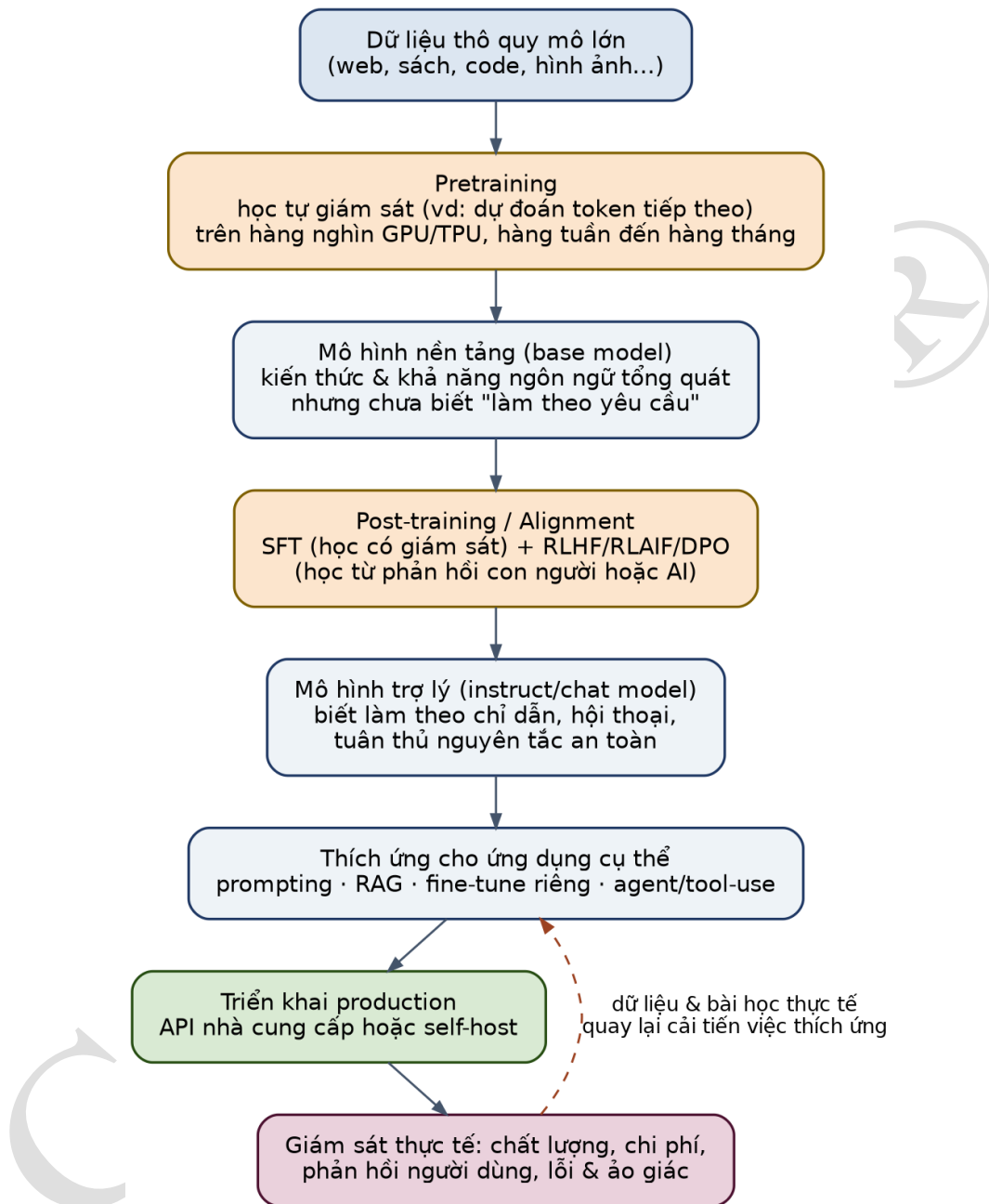
Thuật ngữ "foundation model" được Trung tâm Nghiên cứu Mô hình Nền tảng (CRFM) của Đại học Stanford đề xuất năm 2021, để mô tả một lớp mô hình mới: được huấn luyện trên dữ liệu quy mô rất lớn (thường bằng phương pháp tự giám sát self-supervised learning), rồi có thể thích ứng (adapt) cho rất nhiều tác vụ hạ nguồn khác nhau, thay vì chỉ giải quyết một bài toán hẹp như các mô hình học máy truyền thống.

Ba đặc điểm phân biệt một foundation model với một mô hình machine learning thông thường:

- Quy mô: hàng tỷ đến hàng ngàn tỷ tham số, huấn luyện trên hàng ngàn tỷ token dữ liệu đòi hỏi hạ tầng tính toán mà chỉ một số ít tổ chức trên thế giới sở hữu.
- Tính tổng quát: cùng một mô hình gốc có thể được dùng cho dịch thuật, viết code, tóm tắt, trả lời câu hỏi, phân loại cảm xúc... mà không cần huấn luyện lại từ đầu.
- Học tự giám sát: mô hình học từ chính cấu trúc của dữ liệu (ví dụ dự đoán từ tiếp theo trong câu) thay vì cần con người gán nhãn thủ công cho từng ví dụ đây là lý do vì sao có thể huấn luyện trên gần như toàn bộ văn bản công khai trên Internet.

Kiến trúc đứng sau phần lớn các foundation model ngôn ngữ hiện nay là Transformer, giới thiệu trong bài báo "Attention Is All You Need" (Vaswani và cộng sự, 2017). Cơ chế cốt lõi của Transformer self-attention cho phép mô hình cân nhắc mối liên hệ giữa mọi cặp từ trong một đoạn văn bản cùng lúc, thay vì xử lý tuần tự từng từ một như các kiến trúc mạng hồi quy

(RNN, LSTM) trước đó. Điều này vừa giúp mô hình nắm bắt ngữ cảnh xa tốt hơn, vừa cho phép huấn luyện song song trên phần cứng GPU/TPU hiệu quả hơn nhiều yếu tố quyết định để mở rộng quy mô mô hình lên hàng trăm tỷ tham số.



Hình 1. Vòng đời của một foundation model: từ dữ liệu thô, qua pretraining, alignment, đến khi được thích ứng và triển khai cho ứng dụng thực tế kèm vòng phản hồi liên tục từ production.

2. Vòng đời: từ pretraining đến trợ lý biết nghe lời

2.1. Pretraining giai đoạn tốn kém nhất

Pretraining là giai đoạn mô hình học từ dữ liệu thô quy mô lớn hàng ngàn tỷ token thu thập từ web, sách, mã nguồn, và ngày càng nhiều hình ảnh, âm thanh, video đối với các mô hình đa

phương thức (multimodal). Mục tiêu huấn luyện phổ biến nhất cho mô hình ngôn ngữ là next-token prediction: cho mô hình xem một đoạn văn bản, che đi phần cuối, và yêu cầu nó dự đoán từ (hay chính xác hơn là token) tiếp theo. Nghe đơn giản, nhưng để dự đoán tốt, mô hình buộc phải học được ngữ pháp, sự kiện thế giới, suy luận logic cơ bản, thậm chí một phần kỹ năng lập trình tất cả chỉ từ việc cố gắng đoán đúng từ tiếp theo hàng nghìn tỷ lần.

Chi phí của giai đoạn này rất lớn: có thể lên tới hàng chục đến hàng trăm triệu đô-la tính toán cho các mô hình lớn nhất, chạy trên cụm hàng chục ngàn GPU/TPU liên tục trong nhiều tuần đến nhiều tháng. Đây là lý do vì sao chỉ một số ít tổ chức các phòng thí nghiệm AI lớn và một số công ty công nghệ hàng đầu có khả năng huấn luyện foundation model từ đầu; phần lớn các công ty còn lại (kể cả nhiều công ty AI) chỉ sử dụng lại các mô hình đã được huấn luyện sẵn.

2.2. Post-training / Alignment dạy mô hình "biết nghe lời"

Một mô hình vừa pretrain xong (thường gọi là base model) đã có kiến thức ngôn ngữ rất rộng, nhưng chưa có xu hướng trả lời theo định dạng hội thoại, chưa biết từ chối yêu cầu có hại, và có thể lan man thay vì đi thẳng vào điều người dùng cần. Giai đoạn post-training giải quyết việc này qua hai bước chính:

- Supervised Fine-Tuning (SFT): huấn luyện tiếp trên một tập dữ liệu nhỏ hơn nhiều nhưng có chất lượng cao các cặp hội thoại mẫu do con người viết hoặc chọn lọc để mô hình học đúng "giọng điệu" của một trợ lý hữu ích.
- Học từ phản hồi (RLHF/RLAIF/DPO): mô hình được tinh chỉnh thêm dựa trên việc so sánh và xếp hạng nhiều câu trả lời khác nhau cho cùng một câu hỏi do con người đánh giá (RLHF Reinforcement Learning from Human Feedback), do chính một mô hình AI khác đánh giá theo một bộ nguyên tắc cho trước (RLAIF, ví dụ cách tiếp cận "Constitutional AI" mà Anthropic công bố), hoặc bằng các phương pháp tối ưu trực tiếp hơn như DPO (Direct Preference Optimization) không cần huấn luyện một mô hình phần thưởng riêng.

Kết quả của giai đoạn này là một mô hình trợ lý (instruct/chat model) chính là những gì người dùng cuối tương tác khi gọi API hay dùng giao diện chat của các nhà cung cấp mô hình. Việc phân biệt rõ base model và instruct model rất quan trọng với kỹ sư: một số bài toán (ví dụ sinh văn bản tự do, hoàn thành đoạn văn) đôi khi lại phù hợp với base model hơn, trong khi phần lớn ứng dụng thực tế cần instruct model để nhận chỉ dẫn đáng tin cậy.

3. Hai hướng kiến trúc: Dense và Mixture-of-Experts

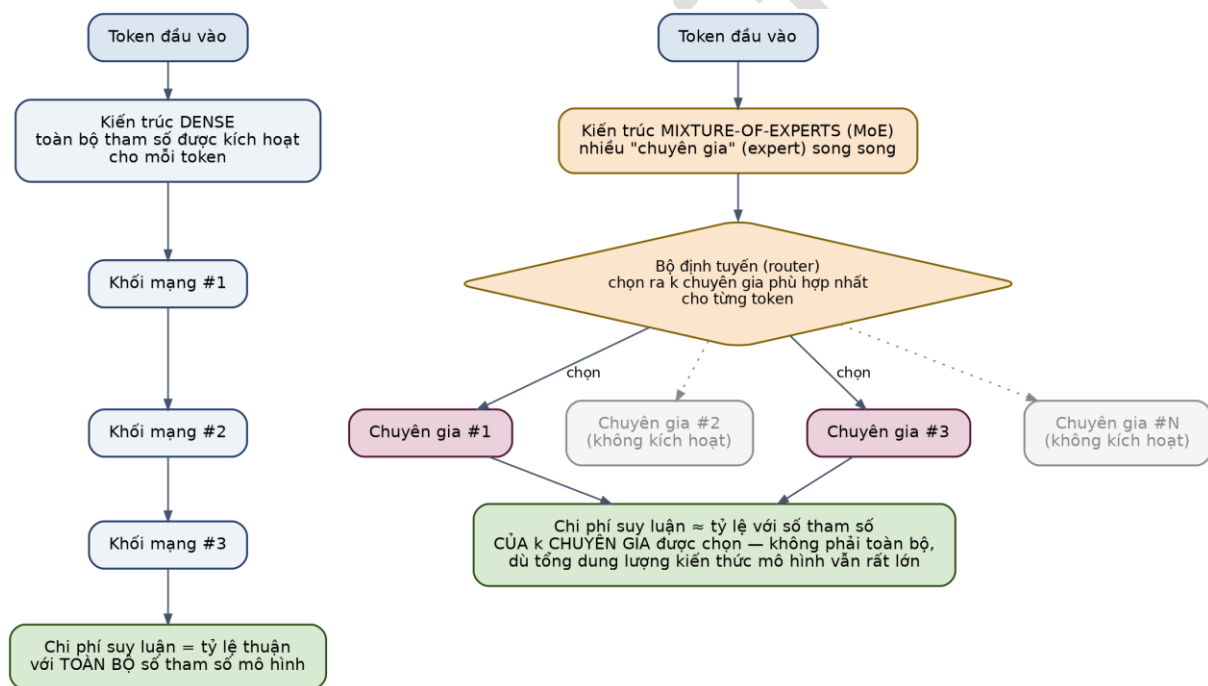
Khi các phòng thí nghiệm AI muốn tăng "dung lượng kiến thức" của mô hình mà không làm chi phí suy luận (inference) tăng theo cùng tỷ lệ, một hướng kiến trúc ngày càng phổ biến là Mixture-of-Experts (MoE) khác với kiến trúc dense truyền thống, nơi mọi tham số của mô hình đều được dùng cho mỗi token đầu vào.

3.1. Dense: đơn giản, dễ dự đoán

Trong một mô hình dense, mỗi token đi qua toàn bộ mạng mọi lớp, mọi tham số đều được kích hoạt. Cách tiếp cận này đơn giản để huấn luyện và phục vụ, chi phí suy luận có thể ước lượng trực tiếp từ tổng số tham số. Nhược điểm là muốn mô hình "biết nhiều hơn", gần như bắt buộc phải tăng số tham số kích hoạt, kéo theo chi phí tính toán tăng tương ứng cho mọi truy vấn kể cả những truy vấn đơn giản không cần đến toàn bộ năng lực đó.

3.2. Mixture-of-Experts: tách dung lượng khỏi chi phí suy luận

Một mô hình MoE thay một số khối mạng dense bằng nhiều "chuyên gia" (expert) song song, cộng thêm một bộ định tuyến (router) nhỏ có nhiệm vụ, với mỗi token, chọn ra chỉ một số ít chuyên gia (thường gọi là top-k) để thực sự tính toán các chuyên gia còn lại không được kích hoạt cho token đó. Nhờ vậy, tổng dung lượng tham số của mô hình có thể rất lớn (giúp mô hình lưu trữ được nhiều kiến thức và kỹ năng chuyên biệt hơn), trong khi chi phí tính toán thực tế cho mỗi token chỉ tỷ lệ với số tham số của các chuyên gia được chọn nhỏ hơn nhiều so với tổng số tham số toàn mô hình.



Hình 2. So sánh kiến trúc Dense và Mixture-of-Experts: cùng một token, dense kích hoạt toàn bộ mạng, còn MoE chỉ kích hoạt một tập con chuyên gia do bộ định tuyến chọn ra.

Đánh đổi của MoE không hề miễn phí: việc huấn luyện phức tạp hơn (cân bằng tải giữa các chuyên gia là một bài toán kỹ thuật không nhỏ), việc phục vụ (serving) đòi hỏi hạ tầng có thể định tuyến động, và toàn bộ tham số của mọi chuyên gia vẫn cần được nạp vào bộ nhớ dù chỉ một phần được dùng mỗi lần nghĩa là MoE tiết kiệm về tính toán (compute) nhưng không tiết kiệm tương ứng về bộ nhớ (memory). Dù vậy, hướng tiếp cận này đã được nhiều mô hình hàng

đầu áp dụng trong vài năm gần đây, vì nó cho phép mở rộng dung lượng mô hình nhanh hơn tốc độ tăng chi phí suy luận.

4. Chọn mô hình cho bài toán thực tế: những trục cần cân nhắc

Với một kỹ sư đang xây dựng ứng dụng, câu hỏi thường gặp nhất không phải "mô hình nào tốt nhất" vì thứ hạng đó thay đổi liên tục mà là "mô hình nào phù hợp nhất với ràng buộc của bài toán này". Dưới đây là những trục lựa chọn ít thay đổi theo thời gian hơn, dùng được lâu dài.

4.1. Closed API và open-weight

Mô hình closed (chỉ truy cập qua API của nhà cung cấp, như các dòng mô hình hàng đầu của OpenAI, Anthropic, hay Google) thường dẫn đầu về chất lượng ở thời điểm ra mắt, không đòi hỏi hạ tầng GPU riêng, và được cập nhật/bảo trì bởi nhà cung cấp. Đổi lại, dữ liệu gửi đi qua API cần tuân theo chính sách của bên thứ ba, chi phí tính theo lượng sử dụng có thể khó dự đoán ở quy mô lớn, và đội ngũ phụ thuộc vào tính sẵn sàng của dịch vụ bên ngoài.

Mô hình open-weight (trọng số mô hình được công bố công khai, có thể tải về và tự vận hành, như các dòng Llama của Meta, Mistral, Qwen, DeepSeek, hay Gemma của Google) cho phép tự lưu trữ (self-host), kiểm soát dữ liệu hoàn toàn trong hạ tầng riêng, và tùy biến sâu hơn (kể cả fine-tune). Đổi lại, đội ngũ phải tự vận hành hạ tầng suy luận, và các mô hình open-weight tốt nhất thường đi sau các mô hình closed hàng đầu một khoảng thời gian nhất định về chất lượng ở những tác vụ khó nhất dù khoảng cách này đã thu hẹp đáng kể qua từng năm.

4.2. Cửa sổ ngữ cảnh (context window) và đa phương thức

Cửa sổ ngữ cảnh lượng văn bản (đo bằng token) mà mô hình có thể "nhìn thấy" cùng lúc trong một lượt gọi đã tăng rất nhanh, từ vài ngàn token của các mô hình đời đầu lên hàng trăm ngàn, thậm chí hàng triệu token ở một số mô hình hiện nay. Cửa sổ ngữ cảnh lớn hữu ích cho các tác vụ cần xử lý tài liệu dài hoặc nhiều tài liệu cùng lúc, nhưng không phải lúc nào cũng thay thế được kiến trúc truy xuất có chọn lọc như RAG nhồi toàn bộ tài liệu vào ngữ cảnh vừa tốn chi phí, vừa có thể khiến mô hình khó tập trung vào đúng phần thông tin liên quan (một hiện tượng đôi khi được gọi là "lost in the middle"). Bên cạnh đó, ngày càng nhiều mô hình hỗ trợ đa phương thức (multimodal) nhận đầu vào là hình ảnh, âm thanh, thậm chí video, không chỉ văn bản mở rộng đáng kể phạm vi ứng dụng có thể xây dựng.

4.3. Chi phí, độ trễ, và "đủ tốt" thay vì "tốt nhất"

Trong thực tế vận hành, một mô hình nhỏ hơn, rẻ hơn, phản hồi nhanh hơn thường là lựa chọn đúng cho phần lớn các bước trong một luồng xử lý chỉ dùng mô hình mạnh nhất (và đắt nhất) cho những bước thực sự cần khả năng suy luận sâu. Đây cũng là một nguyên tắc kiến trúc phổ

biến trong các hệ thống agentic hiện đại: định tuyến mỗi bước tới đúng mô hình phù hợp về năng lực và chi phí, thay vì dùng một mô hình duy nhất cho toàn bộ luồng xử lý.

Một khung hỏi thực dụng khi chọn mô hình: (1) bài toán có cần suy luận nhiều bước phức tạp, hay chỉ là phân loại/trích xuất đơn giản; (2) dữ liệu đầu vào có thuộc diện nhạy cảm, cần kiểm soát nơi lưu trữ và xử lý hay không; (3) khối lượng truy vấn dự kiến ở quy mô production là bao nhiêu, và chi phí cộng dồn theo quy mô đó ra sao; (4) đội ngũ có đủ năng lực vận hành hạ tầng suy luận riêng hay không, nếu chọn hướng self-host.

5. Checklist nhanh cho đội ngũ

- Đã phân biệt rõ nhu cầu dùng base model hay instruct model cho từng tác vụ trong hệ thống chưa?
- Đã cân nhắc đánh đổi giữa API đóng (nhanh triển khai, ít kiểm soát) và open-weight tự host (kiểm soát cao, chi phí vận hành hạ tầng) dựa trên yêu cầu dữ liệu và ngân sách thực tế chưa?
- Có đang dùng một mô hình mạnh nhất, đắt nhất cho mọi bước kể cả những bước đơn giản không cần đến năng lực đó?
- Cửa sổ ngữ cảnh cần dùng có thực sự cần lớn, hay một kiến trúc truy xuất có chọn lọc (RAG) sẽ hiệu quả và rẻ hơn?
- Có kế hoạch theo dõi (evaluation) chất lượng đầu ra theo thời gian, để phát hiện sớm khi nào cần đổi mô hình hoặc điều chỉnh cách dùng?

Đọc thêm

Tài liệu này chỉ là một bản đồ tổng quan. Nếu bạn muốn đào sâu hơn với một góc nhìn hệ thống, đầy đủ ví dụ mã nguồn và case study thực chiến về xây dựng ứng dụng trên nền foundation model bao gồm cả những phần chưa được đề cập ở đây như huấn luyện dữ liệu, đánh giá mô hình chi tiết, hay thiết kế hệ thống production cuốn sách sau là một nguồn tham khảo chuyên sâu đáng đọc:

"AI Engineering: Building Applications with Foundation Models" Chip Huyen, O'Reilly Media, 2025.

Bản tiếng Việt phát hành bởi Times, có bán tại các nhà sách và sàn thương mại điện tử.

Tài liệu bạn đang đọc là nội dung gốc, biên soạn độc lập dựa trên kiến thức chung của ngành và các nguồn công khai liệt kê bên dưới không phải bản dịch hay tóm tắt của cuốn sách trên.

Tài liệu tham khảo

- Vaswani, A. et al. "Attention Is All You Need." NeurIPS, 2017.
- Bommasani, R. et al. "On the Opportunities and Risks of Foundation Models." Stanford CRFM, 2021.
- Anthropic. "Constitutional AI: Harmlessness from AI Feedback." anthropic.com/research, 2022.
- Anthropic. "Introducing the next generation of Claude." anthropic.com/news, 2024.
- Rafailov, R. et al. "Direct Preference Optimization." NeurIPS, 2023.
- Hugging Face. "Mixture of Experts Explained." huggingface.co/blog/moe.
- Ouyang, L. et al. "Training Language Models to Follow Instructions with Human Feedback (InstructGPT)." OpenAI/NeurIPS, 2022.